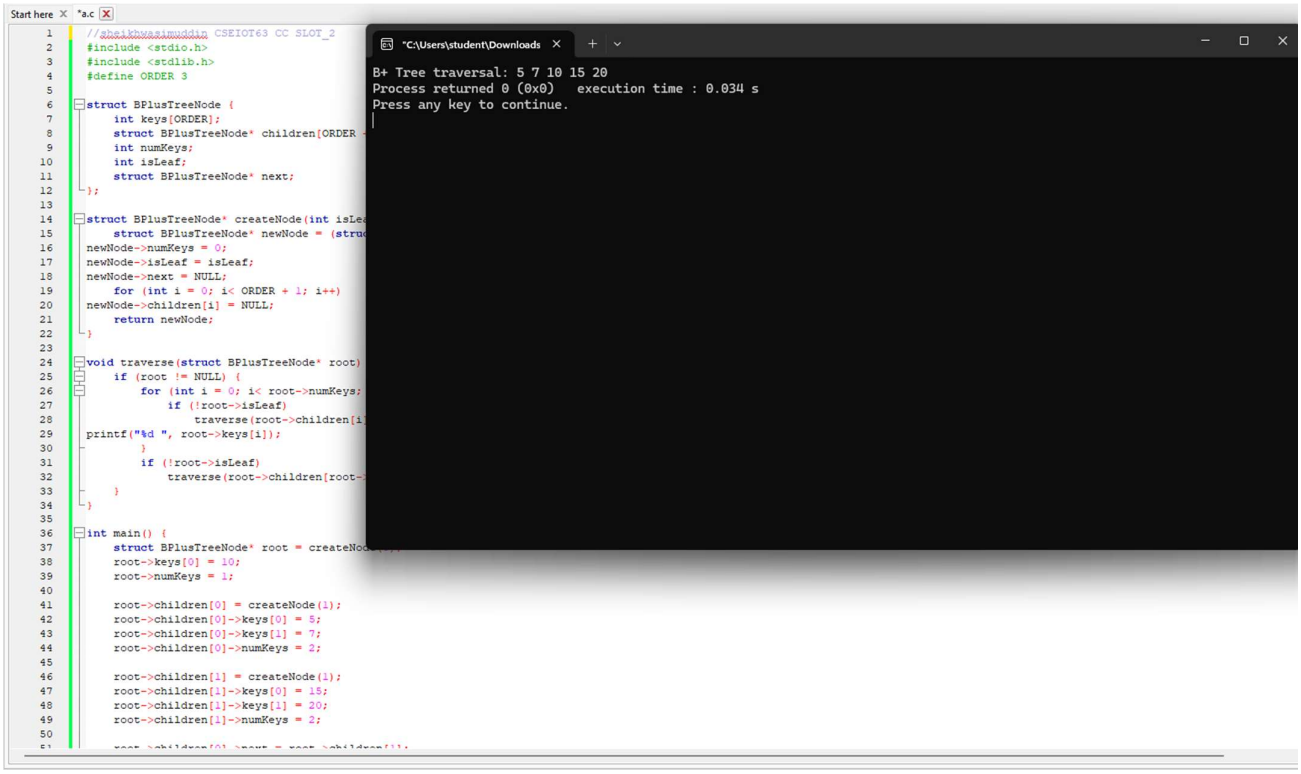
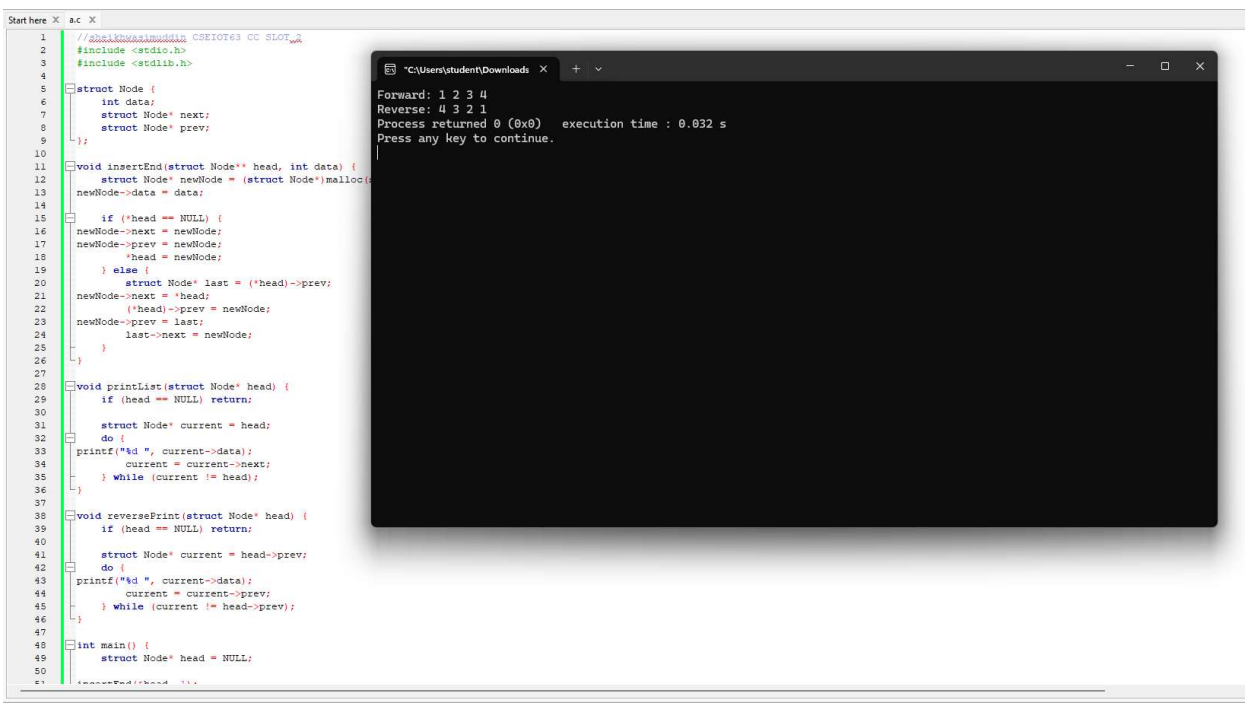
Name: Sheikh Wasimuddin
CSEIOT_63_CC LAB
Enrollment no.: 23071044

1. B+ Tree Operation



2. Circular Doubly Linked List



Name: Sheikh Wasimuddin
CSEIOT_63_CC LAB
Enrollment no.: 23071044

3. Sparse Matrix Representation

Start here | a.c

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#define ROWS 4
#define COLS 5

struct Element {
    int row;
    int col;
    int value;
};

void compressMatrix(int matrix[ROWS][COLS], struct Element sparse[], int* count) {
    *count = 0;
    for (int i = 0; i< ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (matrix[i][j] != 0) {
                sparse[*count].row = i;
                sparse[*count].col = j;
                sparse[*count].value = matrix[i][j];
                (*count)++;
            }
        }
    }
}

void printSparse(struct Element sparse[], int count) {
printf("Row\tCol\tValue\n");
    for (int i = 0; i< count; i++) {
printf("%d\t%d\t%d\n", sparse[i].row, sparse[i].col, sparse[i].value);
    }
}

int main() {
    int matrix[ROWS][COLS] = {
        {0, 0, 3, 0, 4},
        {0, 0, 5, 7, 0},
        {0, 0, 0, 0, 0},
        {0, 2, 6, 0, 0}
    };

    struct Element sparse[ROWS * COLS];
    int count;

compressMatrix(matrix, sparse, &count);
printSparse(sparse, count);

    return 0;
}
```

"C:\Users\student\Downloads

```
Row     Col     Value
0       2       3
0       4       4
1       2       5
1       3       7
3       1       2
3       2       6

Process returned 0 (0x0)   execution time : 0.032 s
Press any key to continue.
```

4. Deque Implementation

Start here | a.c

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#define MAX 5

struct Deque {
    int arr[MAX];
    int front;
    int rear;
    int size;
};

struct Deque* createDeque() {
    struct Deque* deque = (struct Deque*)mall
    deque->front = -1;
    deque->rear = 0;
    deque->size = 0;
    return deque;
}

int isFull(struct Deque* deque) {
    return deque->size == MAX;
}

int isEmpty(struct Deque* deque) {
    return deque->size == 0;
}

void insertFront(struct Deque* deque, int key
    if (isFull(deque)) return;

    if (deque->front == -1) {
        deque->front = 0;
        deque->rear = 0;
    } else if (deque->front == 0)
        deque->front = MAX - 1;
    else
        deque->front = deque->front - 1;

    deque->arr[deque->front] = key;
    deque->size++;
}

void insertRear(struct Deque* deque, int key) {
    if (isFull(deque)) return;
```

"C:\Users\student\Downloads

```
20 15 5 10
15 5

Process returned 0 (0x0)   execution time : 0.030 s
Press any key to continue.
```

Name: Sheikh Wasimuddin
CSEIOT_63_CC LAB
Enrollment no.: 23071044

5. Threaded Binary Tree

Start here | a.c

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
    int leftThread;
    int rightThread;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
newNode->data = data;
newNode->left = newNode->right = NULL;
newNode->leftThread = newNode->rightThread = 0;
    return newNode;
}

struct Node* leftMost(struct Node* node) {
    if (node == NULL) return NULL;
    while (node->left != NULL && node->leftThread == 0)
        node = node->left;
    return node;
}

void inorder(struct Node* root) {
    struct Node* current = leftMost(root);
    while (current != NULL) {
printf("%d ", current->data);

        if (current->rightThread == 1)
            current = current->right;
        else
            current = leftMost(current->right);
    }
}

int main() {
    struct Node* root = createNode(6);
    root->left = createNode(3);
    root->right = createNode(8);
    root->left->left = createNode(1);
    root->left->right = createNode(5);
    root->right->left = createNode(7);
```

"C:\Users\student\Downloads

```
Inorder traversal of threaded binary tree: 1 3 5 6 7 8 11
Process returned 0 (0x0)   execution time : 0.028 s
Press any key to continue.
```

6. Bloom Filter Implementation

Start here | a.c

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 100

struct BloomFilter {
    int bitArray[SIZE];
    int numHashFunctions;
};

unsigned int hash1(const char* str) {
    unsigned int hash = 0;
    while (*str) {
        hash = (hash * 31) + *str;
        str++;
    }
    return hash % SIZE;
}

unsigned int hash2(const char* str) {
    unsigned int hash = 5381;
    while (*str) {
        hash = ((hash << 5) + hash) + *str;
        str++;
    }
    return hash % SIZE;
}

void initFilter(struct BloomFilter* filter) {
memset(filter->bitArray, 0, sizeof(filter->bitArra
    filter->numHashFunctions = 2;
}

void insert(struct BloomFilter* filter, const char* str) {
    int h1 = hash1(str);
    int h2 = hash2(str);

    filter->bitArray[h1] = 1;
    filter->bitArray[h2] = 1;
}

int mightContain(struct BloomFilter* filter, const char* str) {
    int h1 = hash1(str);
```

"C:\Users\student\Downloads

```
Contains 'apple': 1
Contains 'banana': 1
Contains 'orange': 1
Contains 'grape': 0
Contains 'mango': 0

Process returned 0 (0x0)   execution time : 0.039 s
Press any key to continue.
```

Name: Sheikh Wasimuddin
CSEIOT_63_CC LAB
Enrollment no.: 23071044

7. Ternary Search Tree

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#include <stdlib.h>

struct Node {
    char data;
    int isEndOfString;
    struct Node *left, *eq, *right;
};

struct Node* newNode(char data) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->isEndOfString = 0;
    temp->left = temp->eq = temp->right = NULL;
    return temp;
}

void insert(struct Node** root, char* word) {
    if (!(*root))
        *root = newNode(*word);

    if ((*word) < (*root)->data)
        insert(&((*root)->left), word);
    else if ((*word) > (*root)->data)
        insert(&((*root)->right), word);
    else {
        if (*(word + 1))
            insert(&((*root)->eq), word + 1);
        else
            (*root)->isEndOfString = 1;
    }
}

int search(struct Node* root, char* word) {
    if (!root) return 0;

    if (*word < root->data)
        return search(root->left, word);
    else if (*word > root->data)
        return search(root->right, word);
    else {
        if (*(word + 1) == '\0')
            return root->isEndOfString;
        return search(root->eq, word + 1);
```

```
Search for 'cat': 1
Search for 'cats': 1
Search for 'up': 1
Search for 'bu': 0
Search for 'bug': 1

Process returned 0 (0x0)   execution time : 0.037 s
Press any key to continue.
```

8. Segment Tree with Lazy Propagation

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#include <math.h>

int getMid(int s, int e) {
    return s + (e - s)/2;
}

void updateRangeUtil(int *st, int *lazy, int si, int ss, int se, int us, int ue, int diff) {
    if (lazy[si] != 0) {
st[si] += (se - ss + 1) * lazy[si];
        if (ss != se) {
lazy[si*2 + 1] += lazy[si];
lazy[si*2 + 2] += lazy[si];
        }
        lazy[si] = 0;
    }

    if (ss > se || ss >ue || se < us)
        return;

    if (ss >= us && se <= ue) {
st[si] += (se - ss + 1) * diff;
        if (ss != se) {
lazy[si*2 + 1] += diff;
lazy[si*2 + 2] += diff;
        }
        return;
    }

    int mid = getMid(ss, se);
updateRangeUtil(st, lazy, si*2+1, ss, mid, us, ue, diff);
updateRangeUtil(st, lazy, si*2+2, mid+1, se, us, ue, diff);

st[si] = st[si*2+1] + st[si*2+2];
}

void updateRange(int *st, int *lazy, int n, int us, int ue, int diff) {
updateRangeUtil(st, lazy, 0, 0, n-1, us, ue, diff);
}

int getSumUtil(int *st, int *lazy, int si, int ss, int se, int qs, int qe) {
    if (lazy[si] != 0) {
st[si] += (se - ss + 1) * lazy[si];
        if (ss != se) {
lazy[si*2 + 1] += lazy[si];
lazy[si*2 + 2] += lazy[si];
        }
        lazy[si] = 0;
    }
```

```
Sum of values in given range = 15
Updated sum of values in given range = 45

Process returned 0 (0x0)   execution time : 0.044 s
Press any key to continue.
```

Name: Sheikh Wasimuddin
CSEIOT_63_CC LAB
Enrollment no.: 23071044

9. Count Inversions using BIT

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#include <stdlib.h>

int getSum(int BITree[], int index) {
    int sum = 0;
    while (index > 0) {
        sum += BITree[index];
        index -= index & (-index);
    }
    return sum;
}

void updateBIT(int BITree[], int n, int index
    while (index <= n) {
BITree[index] += val;
        index += index & (-index);
    }
}

int getInvCount(int arr[], int n) {
    int invcount = 0;
    int maxElement = 0;

    for (int i = 0; i< n; i++)
        if (maxElement<arr[i])
maxElement = arr[i];

    int BIT[maxElement + 1];
    for (int i = 1; i<= maxElement; i++)
        BIT[i] = 0;

    for (int i = n - 1; i>= 0; i--) {
invcount += getSum(BIT, arr[i] - 1);
updateBIT(BIT, maxElement, arr[i], 1);
    }

    return invcount;
}

int main() {
    int arr[] = {8, 4, 2, 1};
    int n = sizeof(arr)/sizeof(int);

printf("Number of inversions are %d\n", getInvCount(arr, n));
```

```
"C:\Users\student\Downloads
Number of inversions are 6

Process returned 0 (0x0)   execution time : 0.030 s
Press any key to continue.
```

10. Circular Buffer/Ring Buffer

```c
//sheikhwasimuddin CSEIOT63 CC SLOT_2
#include <stdio.h>
#include <stdlib.h>
#define SIZE 5

struct CircularBuffer {
    int buffer[SIZE];
    int head;
    int tail;
    int count;
};

void initBuffer(struct CircularBuffer* cb) {
cb->head = 0;
cb->tail = 0;
cb->count = 0;
}

int isFull(struct CircularBuffer* cb) {
    return cb->count == SIZE;
}

int isEmpty(struct CircularBuffer* cb) {
    return cb->count == 0;
}

void enqueue(struct CircularBuffer* cb, int data) {
    if (isFull(cb)) {
cb->head = (cb->head + 1) % SIZE;
cb->count--;
    }

cb->buffer[cb->tail] = data;
cb->tail = (cb->tail + 1) % SIZE;
cb->count++;
}

int dequeue(struct CircularBuffer* cb) {
    if (isEmpty(cb))
        return -1;

    int data = cb->buffer[cb->head];
cb->head = (cb->head + 1) % SIZE;
cb->count--;
    return data;
```

```
"C:\Users\student\Downloads
After enqueue 10: 10
After enqueue 20: 10 20
After enqueue 30: 10 20 30
After enqueue 40: 10 20 30 40
After enqueue 50: 10 20 30 40 50
After enqueue 60: 20 30 40 50 60
After enqueue 70: 30 40 50 60 70

Dequeue: 30
After dequeue: 40 50 60 70
After enqueue 80: 40 50 60 70 80

Process returned 0 (0x0)   execution time : 0.041 s
Press any key to continue.
```